

VisionSearch

A Semantic Image Search Engine

Build & ship roadmap — from-scratch contrastive multimodal retrieval
Scoped for a single NVIDIA RTX 5060 (8 GB VRAM) | Zero-cost stack | Placement-focused

“Type a sentence, get the matching images.”

Goal	A deployed text-to-image search tool backed by a dual-encoder you train yourself
Core skill shown	Contrastive learning (InfoNCE), embeddings & vector retrieval — the foundation of modern RAG
Timeline	8 weeks, part-time student pace (adjust freely)
Cost	Rs. 0 — free datasets, your own GPU, open weights, free hosting
Final artifact	Live demo link + documented GitHub repo with a baseline comparison

1 Why this project (for placement)

At the entry level, recruiters reward a **complete, deployed, well-explained** project far more than a flashy one that does not run. This project is deliberately chosen to win on all three fronts while staying inside an 8 GB memory budget and costing nothing.

- **It is a real product, not a notebook.** A search box anyone can use in ten seconds reads as engineering, not homework.
- **It teaches the in-demand skill.** Embeddings + vector similarity is the literal backbone of every retrieval-augmented (RAG) system companies are building right now.
- **The from-scratch core is genuine and explainable.** You build and train the contrastive alignment loop yourself — the hard, defensible part of multimodal ML.
- **It fits 8 GB.** Retrieval is lighter than text generation, and freezing the heavy image backbone lets you precompute features and train cheaply.
- **It ships.** Free hosting means a clickable link on your resume, which is the single biggest differentiator at this level.

2 What you are building

A **dual-encoder** model. One encoder turns an image into a vector; another turns a text query into a vector. Both land in the **same shared embedding space**, trained so that a caption sits close to its image and far from everyone else's. At search time you embed the query once and rank the image library by cosine similarity.

TEXT QUERY		IMAGE LIBRARY
"two people on a beach at sunset"		(thousands of images)
Text encoder + projection head		Image encoder (frozen) + projection head
↓ text vector		↓ image vectors (precomputed, in FAISS index)

The part that is yours ("from scratch"): the projection heads, the contrastive training loop, the symmetric InfoNCE loss, and the learnable temperature. You start from pretrained backbones — which is exactly what real vision-language models do — and train the alignment. Knowing *not* to pretrain a backbone on a laptop is itself a senior-level signal; say so in your README.

3 Zero-cost tech stack

Layer	Tool	Why / note
Framework	PyTorch	Industry standard; CUDA support for your 5060
Backbones	Hugging Face / timm (open weights)	e.g. image: ViT-B/32 or ResNet-50; text: DistilBERT

Dataset	Flickr30k (free, Kaggle/HF)	~31k images, 5 captions each — ideal size for 8 GB
Vector search	FAISS (CPU)	Fast similarity search over precomputed image vectors
Experiment logs	TensorBoard or W&B free tier	Loss/recall curves — screenshots go in your README
UI	Gradio or Streamlit	A search box and a results grid in ~50 lines
Hosting	Hugging Face Spaces / Streamlit Cloud	Free public demo link for your resume
Version control	Git + GitHub	Clean commit history is itself a hiring signal

4 Making it fit in 8 GB

Contrastive learning loves **large batches** (more in-batch negatives = a sharper signal), and large batches are exactly what 8 GB resists. These five moves resolve that tension and are themselves the “efficient training” story you tell in interviews:

- **Freeze the image backbone and precompute image features once.** The biggest win. No image-side gradients or optimizer state, and a frozen encoder means you embed every image a single time and reuse it — training becomes light and fast.
- **Mixed precision (AMP, fp16/bf16).** Roughly halves activation memory and speeds up the tensor cores on Blackwell.
- **Gradient accumulation.** Simulates a big batch by summing several small forward/backward passes before stepping.
- **Keep images at 224×224.** Resolution drives memory quadratically; resist the urge to go higher.
- **Train the text encoder with LoRA (optional).** If you fine-tune the text side, parameter-efficient tuning keeps optimizer memory tiny.

Component	Setting	Approx. VRAM mindset
Image encoder	Frozen, features precomputed	~0 during training (offline pass only)
Text encoder	DistilBERT, fp16, LoRA or full FT	Largest live consumer — keep batch modest
Projection heads	Trained from scratch	Tiny
Effective batch	e.g. 32 real × 4 accum = 128	Tune down first if you hit OOM

Note on the trade-off: freezing the image backbone is the safe, fast default. If you want a deeper result, unfreeze it as a *stretch goal* (Week 8+) using the gradient-cache technique for large-batch contrastive learning on limited memory. Start frozen; earn the unfreeze.

5 Week-by-week roadmap

Week 1 — Foundations & setup

- Install PyTorch with CUDA; run a tensor on the GPU to confirm the 5060 is visible.
- Create the GitHub repo + a clean folder structure (data/, src/, notebooks/, app/).
- Read the CLIP paper and write a one-page note in your own words on contrastive learning, dual encoders, and the InfoNCE loss.
- **Deliverable:** working environment, repo skeleton, theory note committed.

Week 2 — Data pipeline

- Download Flickr30k; inspect the image/caption structure.
- Build a PyTorch Dataset + DataLoader: image transforms (resize to 224, normalize), caption tokenization.
- Make a clean train / val / test split with no caption leakage across splits.
- **Deliverable:** a notebook that loads a batch and visualizes images with their captions.

Week 3 — Model architecture

- Load the pretrained image backbone (frozen) and text encoder.
- Write the two projection heads that map both encoders to the same dimension (e.g. 256/512).
- Implement the dual-encoder forward pass; add shape-assertion tests.
- **Deliverable:** model.py with a passing forward pass on one batch.

Week 4 — Contrastive training loop

- Implement the symmetric InfoNCE loss with a learnable temperature.
- Add AMP mixed precision, gradient accumulation, and logging (TensorBoard/W&B).
- Sanity check: overfit a tiny subset (loss should crash toward zero).
- **Deliverable:** training script + a screenshot of a falling loss curve.

Week 5 — Full training & tuning

- Train on the full dataset; tune learning rate, temperature, batch/accumulation.
- Handle OOM methodically: lower batch first, then sequence length, then resolution.
- Checkpoint regularly; keep the best by validation recall.
- **Deliverable:** a trained checkpoint + logged runs you can point to.

Week 6 — Evaluation

- Compute Recall@1 / @5 / @10 for text-to-image (and image-to-text) on the test set.
- **Run the baseline:** measure raw pretrained CLIP on the same test set and compare. This contrast is your strongest interview evidence.
- Collect qualitative wins and failure cases (good and bad query examples).
- **Deliverable:** an evaluation report with a metrics table + example results.

Week 7 — Build the search app

- Precompute embeddings for a demo gallery; build a FAISS index for fast ranking.
- Build the Gradio/Streamlit UI: a query box that returns the top-k images.
- Add a couple of nice-to-haves: example queries, similarity scores on results.
- **Deliverable:** a working app running locally end to end.

Week 8 — Deploy & document

- Deploy to Hugging Face Spaces / Streamlit Cloud; confirm the public link works.
- Write the README: problem, architecture diagram, design decisions, results table vs baseline, demo GIF, how-to-run.
- Add the “interview talking points” (next page) to your own notes.
- **Deliverable:** live demo link + a polished, recruiter-ready repo.

6 How you measure success

Retrieval is judged by **Recall@K**: for a given text query, does the correct image appear in the top K results? Report it for K = 1, 5, 10. Always show your model **next to the off-the-shelf baseline** so the number means something.

Metric	Reads as
Recall@1	Correct image is the very top result
Recall@5	Correct image is somewhere in the top 5
Recall@10	Correct image is in the top 10
Baseline delta	Your model vs raw pretrained CLIP on the same test set

7 Interview talking points

Prepare a one-line answer for each. These are what panels actually probe:

- **Dual-encoder vs cross-encoder?** Dual encoders let you precompute image vectors and search millions instantly; cross-encoders are accurate but too slow to scale to retrieval.
- **What does InfoNCE do?** Pulls each image-caption pair together while pushing apart all other pairs in the batch — the in-batch negatives.
- **Why a temperature parameter?** It scales the similarity logits, controlling how sharply the model separates positives from negatives.
- **Why freeze the image backbone?** Memory + data efficiency on 8 GB, and it mirrors how real VLMs train the alignment, not the encoder.
- **How does search scale?** FAISS indexes the precomputed vectors so top-k lookup stays fast as the library grows.
- **What did 8 GB force you to do?** Mixed precision, gradient accumulation, feature precomputation — concrete efficiency engineering.

8 Pitfalls to avoid

- **Leaving it undeployed.** A live link beats a better model that only runs on your laptop. Deployment is non-negotiable.
- **Skipping the baseline comparison.** Without “vs raw CLIP” your numbers are uninterpretable and far less convincing.
- **Tiny batches with no accumulation.** Contrastive learning starves without enough negatives — use accumulation.
- **Caption leakage across splits.** The same image/caption in train and test inflates your scores and is an interview red flag.
- **Scope creep.** Resist adding captioning, VQA, or document understanding now. Ship retrieval first; list the rest as future work.
- **Cranking up image resolution.** The fastest way to OOM on 8 GB for little gain.

9 Stretch goals (after it ships)

- Unfreeze the image backbone using the gradient-cache technique for large-batch contrastive training on limited VRAM.
- Add image-to-image search (“find more like this”) — reuses the same index, near-zero extra work.
- Try a larger or multilingual text encoder so queries work in Indian languages — a genuinely distinctive add-on.
- Repurpose the embeddings for zero-shot image classification and report the accuracy.
- Swap Flickr30k for a COCO subset and show how scores move with more data.

Build order in one line: **environment** → **data** → **model** → **loss/training** → **evaluate vs baseline** → **app** → **deploy** → **document**. Finish each before starting the next. A shipped, documented, baseline-beating retrieval engine is a strong placement project.